Predicting and Pre-empting Retry-Storm-Induced Livelock in Microservice Architectures

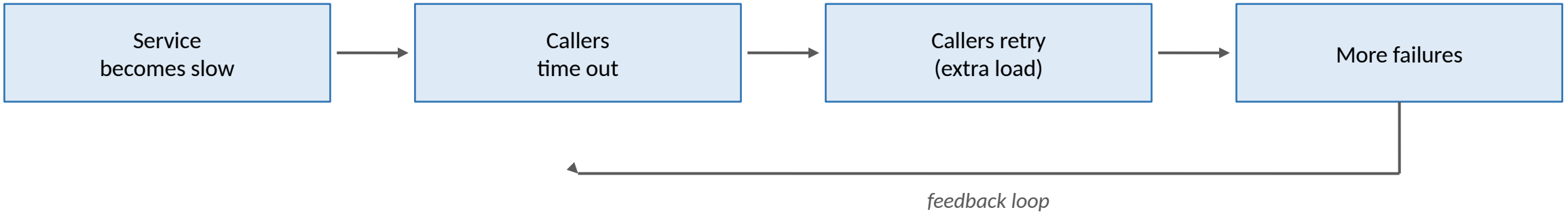
Review 2

BCSE303L - Operating Systems
Vellore Institute of Technology, Vellore

Varun G A (24BCE0479)
Satwik Som (24BCE2822)

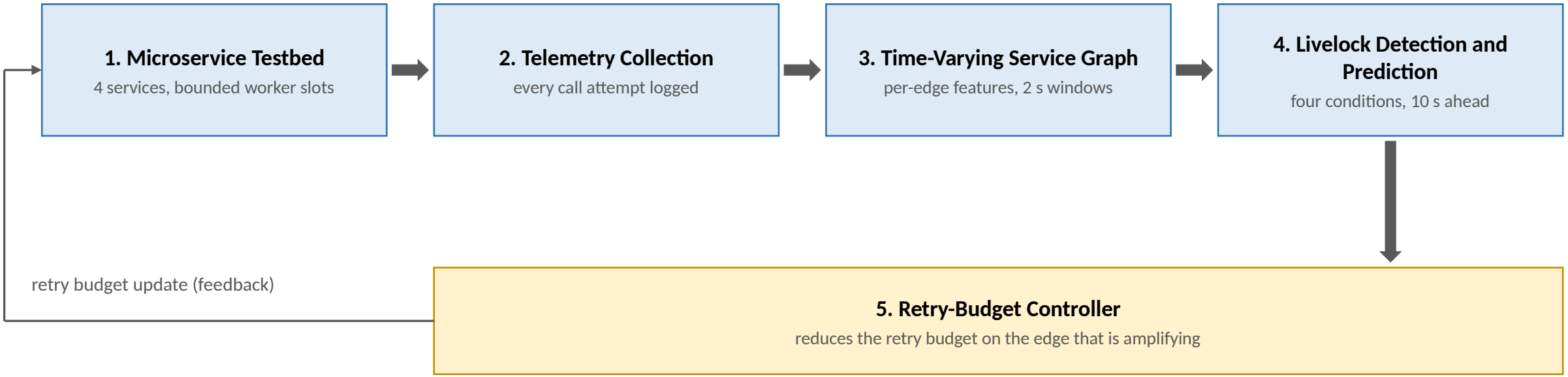
The Problem

- When a microservice becomes slow, every service calling it starts retrying.
- Those retries are extra load on a service that is already struggling.
- More requests fail, so more retries are sent. The loop feeds itself.
- The system stays fully busy but stops completing useful work.
- This is livelock: nothing is blocked, everything is running, no progress is made.



System Design and Architecture

The system is a closed loop. The control action changes the state that is observed next.



Stages 1 to 3 produce the data. Stage 4 decides whether a call edge is heading into livelock.

Stage 5 acts on that decision, and the effect appears in the next round of telemetry.

Related Work

Retry-induced failure is a known problem. This is what the published work covers.

Work	What it contributes	What it does not do
Bronson et al. HotOS 2021	Named the failure class: a trigger pushes the system into a bad state that persists after the trigger is gone	Conceptual only. No metric, no detector, no experiment
Huang et al. OSDI 2022	Studied real incident reports from 11 organisations. Retries are the sustaining cause in over half of them	Describes incidents after the fact. No online detection or prediction
Isaacs et al. HotOS 2025	Queueing models and simulation to find configurations that will fail before they are deployed	Offline and pre-deployment. States that online prediction is still an open problem
Gan et al. (Seer) ASPLOS 2019	Deep learning predicts QoS violations about 100 ms ahead using per-service queue depths	Does not model retries at all. Reacts by resizing containers, not by changing retry policy
Google SRE 2016	Retry budget: a client may retry only while retries stay under 10 percent of its requests	Fixed threshold and reactive. Same budget applied no matter which edge is amplifying
Mehan and Saluja arXiv 2026	Defines a Retry Amplification Factor and an adaptive retry budget, tested in simulation	Analysis and simulation. No ahead-of-time prediction and no definition of livelock
Luo et al. (Alibaba) SoCC 2021	Public trace of about 20,000 microservices and over 10 billion calls over 7 days	Has no per-attempt field, so retry amplification cannot be measured from it

Research Gap and Our Contribution

No existing work predicts retry-induced livelock ahead of time and then acts on the retry policy itself.

Capability	Metastable failure studies	Seer	SRE retry budget	Ours
Measures retry amplification per call edge	No	No	Indirectly	Yes
Test that separates livelock from ordinary overload	No	No	No	Yes
Predicts before the bad state is reached	No	Yes, but for QoS	No	Yes
Acts by changing the retry policy itself	Suggested	No	Yes	Yes
Control driven by a measured per-edge risk score	No	No	No	Yes
Checked against a shuffled-label control run	No	No	No	Yes

We are not claiming to handle failures better than a production system at Google or Meta.

The contribution is the combination: retry amplification measured per call edge from per-attempt telemetry, a four-condition test that tells livelock apart from overload that is recovering, and a controller that changes the retry budget from that signal before the system reaches the bad state.

Implemented Testbed Architecture

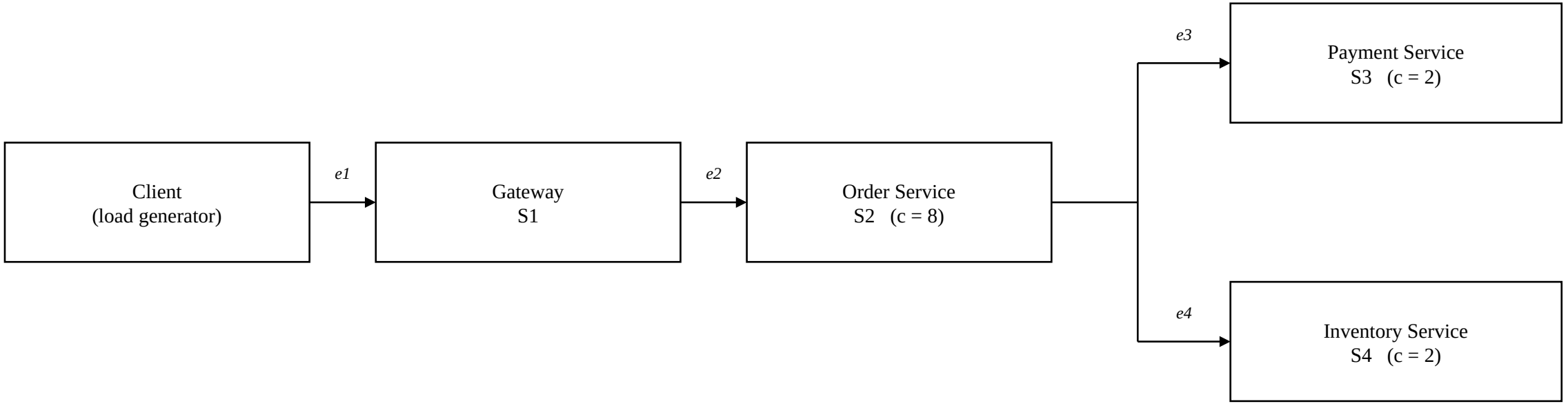


Fig. 2. Implemented microservice testbed. Each service Si has a bounded concurrency limit c; each edge ei retries on failure and is instrumented.

- Every internal call retries on failure, up to 3 attempts with exponential backoff.
- Worker slots are limited, so retries compete with new requests for the same capacity.
- This is what makes the retry storm cause real harm instead of being only extra traffic.
- One user request can become 9 calls to a failing service (3 retries at two levels).

Module Identification

Module	Function
1. Microservice Testbed	Services communicating over HTTP in a call chain
2. Retry and Fault Injection	Inject latency or failures, retry with backoff
3. Telemetry Pipeline	Record every call attempt into structured files
4. Livelock Detection	Apply the four conditions and label time windows
5. Graph Representation	Build a time-varying service call graph
6. Prediction Model	Predict livelock risk a few seconds ahead
7. Retry Controller	Reduce retry budget on a high-risk path

Module Completion

Module	Status	Evidence
1. Microservice Testbed	Completed	4 services run and communicate
2. Retry and Fault Injection	Completed	Retry storm reproduced on demand
3. Telemetry Pipeline	Completed	13,214 call records collected
4. Livelock Detection	Completed	8 windows detected, 0 false alarms
5. Graph Representation	Completed	Graph snapshots exported per window
6. Prediction Model	Completed	F1 0.55, verified against a control test
7. Retry Controller	Completed	Wasted work reduced by 28 percent

Implementation Details

Language and libraries

Python 3, Flask for the services, scikit-learn for the model, NetworkX for the graph.

Fault injection

Each service reads a small config file per request, so faults can be changed while the system runs.

Telemetry recorded for every call

Timestamp, source service, destination service, attempt number, backoff delay, response code, latency.

Two separate logs

One for internal call attempts (how hard the system is working) and one for user requests (what users actually got).

> *The two separate logs are what allow livelock to be shown: high activity together with near-zero progress.*

Livelock Detection Logic

A time window is labelled livelock only if all four conditions hold for two windows in a row.

Condition	Meaning
1. Retry amplification is high	More retry calls than fresh calls on the edge
2. Useful throughput collapsed	Edge delivers less than half its healthy rate
3. Concurrency is not draining	Work in flight stays high instead of clearing
4. Positive feedback is present	Retries not falling while success rate is not improving

Retry amplification: rho = retry calls per second / fresh calls per second

- Healthy traffic gives rho near 0. During a storm we measured rho between 1.4 and 4.5.
- Condition 4 is what separates livelock from ordinary overload. An overloaded system that is recovering fails this condition.
- Requiring all four together is why we got zero false alarms on healthy traffic.

Results and Analysis: Does Retrying Help?

The same fault was applied three times, changing only the retry policy.

Retry policy	Success (normal)	Success (during fault)	Calls made	Avg latency
No retries	88.5 %	0.7 %	475	1337 ms
Always retry (3 attempts)	100.0 %	1.4 %	1036	3473 ms
Adaptive controller	99.4 %	1.7 %	747	2353 ms

- Retrying during the fault gave no extra useful output, but more than doubled the work (475 to 1036 calls) and made users wait 2.6 times longer.
- Turning retries off is not a solution either. Without retries, normal traffic drops from 100 percent to 88.5 percent.
- The adaptive controller keeps success at 99.4 percent when healthy and still cuts wasted work by about 28 percent when the system is struggling.

Results and Analysis: Detection and Prediction

Detection

- 6 test runs, 13,214 call records, 660 time windows analysed.
- 8 windows labelled as livelock.
- 5 in the sustained degraded period, 3 during the ramps into it.
- 0 false alarms during healthy traffic.

Prediction

Predicting livelock 4 to 12 seconds ahead:

	Precision	Recall	F1
Our model	0.38	1.00	0.55
Shuffled labels (control)	0.06	0.60	0.12

- The control test trains the same model on random labels. It scores at chance level (F1 0.12 against a chance level of 0.05), which shows our model is learning real signal and not leaking the answer.
- Recall 1.00 means no coming livelock was missed. Precision 0.38 means some warnings were false alarms.
- For an early warning system this trade is acceptable, because a missed storm costs far more than an unnecessary caution.

Output: Services Running and Telemetry Collected

Terminal 1 - starting the four services

```
$ python3 run_all.py
starting payment_service on port 5001
starting inventory_service on port 5002
starting order_service on port 5003
starting gateway on port 5000
payment_service is up
inventory_service is up
order_service is up
gateway is up
all services started. press Ctrl+C to stop.

$ curl "http://127.0.0.1:5000/checkout?scenario_id=verify&run_type=normal"
checkout complete
```

Four services start, each with its worker slot count

```
python3 analyze_run.py demo1
```

phase	requests	success%	avg_latency	calls	retry%
normal	139	100.0%	181ms	434	4%
degrading	276	29.7%	3227ms	1172	43%
degraded_hold	193	0.5%	3577ms	738	59%
recovery	111	100.0%	200ms	344	3%

Per-phase summary of one gradual-degradation run

telemetry_log.jsonl - one record per call attempt

sample records from telemetry_log.jsonl (one per call attempt)


```
{"timestamp": 1790108347.4123416, "source_service": "order_service", "dest_service": "inventory_service",
    "attempt_number": 2, "backoff_delay_ms": 0, "response_code": 200, "latency_ms": 56.494951248168945,
    "retry_policy": "base_100ms_max_3_timeout_2000ms", "scenario_id": "arm_always_retry", "run_type": "normal"}
{"timestamp": 1790108348.1138628, "source_service": "order_service", "dest_service": "inventory_service",
    "attempt_number": 2, "backoff_delay_ms": 0, "response_code": 200, "latency_ms": 95.72553634643555,
    "retry_policy": "base_100ms_max_3_timeout_2000ms", "scenario_id": "arm_always_retry", "run_type": "normal"}
{"timestamp": 1790108348.543891, "source_service": "gateway", "dest_service": "order_service", "attempt_number":
    2, "backoff_delay_ms": 0, "response_code": 200, "latency_ms": 150.95782279968262, "retry_policy":
    "base_100ms_max_3_timeout_2000ms", "scenario_id": "arm_always_retry", "run_type": "normal"}
{"timestamp": 1790108348.6431162, "source_service": "order_service", "dest_service": "payment_service",
    "attempt_number": 2, "backoff_delay_ms": 0, "response_code": 200, "latency_ms": 43.607473373413086,
    "retry_policy": "base_100ms_max_3_timeout_2000ms", "scenario_id": "arm_always_retry", "run_type": "normal"}
{"timestamp": 1790108348.9395237, "source_service": "order_service", "dest_service": "payment_service",
    "attempt_number": 2, "backoff_delay_ms": 0, "response_code": 200, "latency_ms": 82.54718780517578,
    "retry_policy": "base_100ms_max_3_timeout_2000ms", "scenario_id": "arm_always_retry", "run_type": "normal"}
{"timestamp": 1790108349.4249525, "source_service": "order_service", "dest_service": "inventory_service",
    "attempt_number": 2, "backoff_delay_ms": 0, "response_code": 200, "latency_ms": 107.15150833129883,
    "retry_policy": "base_100ms_max_3_timeout_2000ms", "scenario_id": "arm_always_retry", "run_type": "normal"}
{"timestamp": 1790108349.5259328, "source_service": "order_service", "dest_service": "inventory_service",
    "attempt_number": 2, "backoff_delay_ms": 0, "response_code": 200, "latency_ms": 73.77958297729492,
    "retry_policy": "base_100ms_max_3_timeout_2000ms", "scenario_id": "arm_always_retry", "run_type": "normal"}
{"timestamp": 1790108350.2353542, "source_service": "order_service", "dest_service": "payment_service",
    "attempt_number": 2, "backoff_delay_ms": 0, "response_code": 200, "latency_ms": 39.79802131652832,
    "retry_policy": "base_100ms_max_3_timeout_2000ms", "scenario_id": "arm_always_retry", "run_type": "normal"}
{"timestamp": 1790108350.5407758, "source_service": "order_service", "dest_service": "inventory_service",
    "attempt_number": 2, "backoff_delay_ms": 0, "response_code": 200, "latency_ms": 70.61600685119629,
    "retry_policy": "base_100ms_max_3_timeout_2000ms", "scenario_id": "arm_always_retry", "run_type": "normal"}
{"timestamp": 1790108351.1731827, "source_service": "order_service", "dest_service": "inventory_service",
    "attempt_number": 2, "backoff_delay_ms": 0, "response_code": 200, "latency_ms": 90.29150009155273,
    "retry_policy": "base_100ms_max_3_timeout_2000ms", "scenario_id": "arm_always_retry", "run_type": "normal"}
{"timestamp": 1790108352.210381, "source_service": "order_service", "dest_service": "inventory_service",
    "attempt_number": 2, "backoff_delay_ms": 0, "response_code": 200, "latency_ms": 75.85453987121582,
    "retry_policy": "base_100ms_max_3_timeout_2000ms", "scenario_id": "arm_always_retry", "run_type": "normal"}
{"timestamp": 1790108352.352305, "source_service": "order_service", "dest_service": "payment_service",
    "attempt_number": 2, "backoff_delay_ms": 200, "response_code": 500, "latency_ms": 38.018226623535156,
    "retry_policy": "base_100ms_max_3_timeout_2000ms", "scenario_id": "arm_always_retry", "run_type": "normal"}
```


Raw records showing retry attempts

Output: Livelock Detection

```
python3 labeling.py
```

```
loaded 13214 telemetry records
found 3 edges: [('order_service', 'payment_service'), ('order_service', 'inventory_service'), ('gateway',
    'order_service')]
wrote 660 labeled windows to /sessions/rcw-013refczteplr37byjkeq9cx/work/labeled_windows.csv (8 labeled
    livelock)
```

labeled_windows.csv - the four conditions per window

phase	rho	retries/s	useful/s	concur	c1	c2	c3	c4	label
normal	0.04	0.5	14.0	0.92	F	F	F	T	normal
normal	0.0	0.0	13.5	0.89	F	F	F	F	normal
normal	0.04	0.5	12.5	0.89	F	F	F	T	normal
degrading	0.06	0.5	8.0	6.46	F	F	T	T	normal
degrading	0.18	1.5	8.5	7.08	F	F	T	T	normal
degrading	0.06	0.5	8.5	7.2	F	F	T	F	normal
degrading	0.08	0.5	6.0	4.55	F	T	F	F	normal
degrading	0.08	0.5	5.0	7.29	F	T	T	T	normal
degrading	0.62	2.5	4.5	7.35	F	T	T	T	normal
degrading	0.2	1.0	6.0	7.19	F	T	F	F	normal
degrading	0.2	1.0	5.0	5.48	F	T	F	F	normal
degrading	0.0	0.0	3.0	7.43	F	T	T	F	normal
degrading	1.0	3.0	4.0	7.32	F	T	T	T	normal
degrading	0.33	1.5	4.0	7.42	F	T	F	F	normal
degrading	0.25	1.0	4.0	7.33	F	T	T	F	normal
degrading	1.0	2.0	3.0	5.41	F	T	F	F	normal
degrading	0.27	1.5	2.5	7.13	F	T	F	T	normal
degrading	1.5	3.0	3.0	7.53	T	T	T	T	normal
degrading	0.5	2.5	3.5	7.21	F	T	T	F	normal

Conditions switching on one by one as the service degrades

detection accuracy check

```
total windows analysed : 660
labelled livelock      : 8

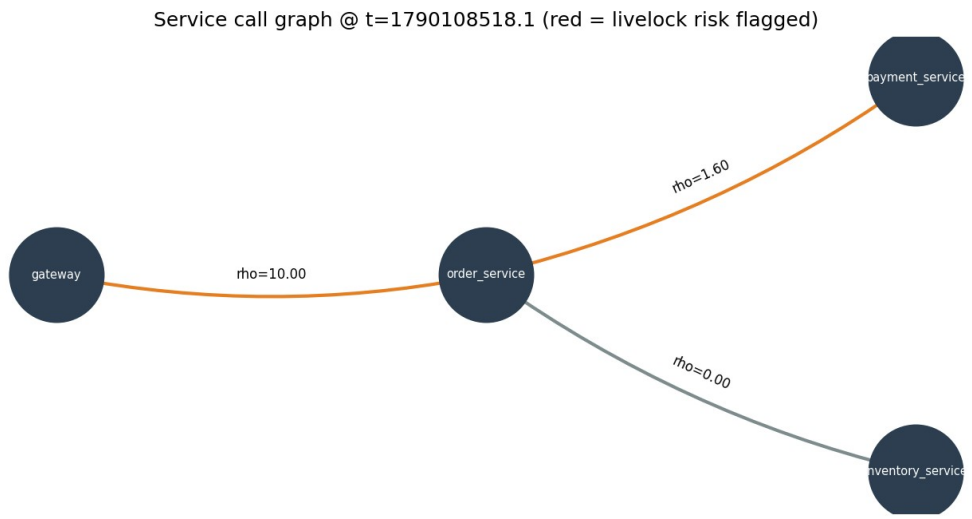
ground truth of the livelock windows:
   degraded_hold    5
   spike            2
   degrading        1

false alarms during healthy traffic: 0

edges these were found on:
   gateway -> order_service                  8
```

Checked against the injected fault

Output: Service Graph and Controller Action



Red edges are the ones flagged as livelock risk

controller.log - retry budget adjusted live

```
controller.py decisions on the order_service -> payment_service edge
(rho is the retry amplification it measures; max_attempts is the budget it sets)

t=1790108416  order_service->payment_service   rho=0.92  ->  max_attempts=3 backoff_base_ms=100
t=1790108418  order_service->payment_service   rho=0.57  ->  max_attempts=3 backoff_base_ms=100
t=1790108420  order_service->payment_service   rho=0.77  ->  max_attempts=3 backoff_base_ms=100
t=1790108422  order_service->payment_service   rho=1.20  ->  max_attempts=2 backoff_base_ms=150
t=1790108424  order_service->payment_service   rho=1.09  ->  max_attempts=1 backoff_base_ms=200
t=1790108426  order_service->payment_service   rho=1.18  ->  max_attempts=1 backoff_base_ms=250
t=1790108428  order_service->payment_service   rho=0.27  ->  max_attempts=2 backoff_base_ms=200
t=1790108430  order_service->payment_service   rho=0.02  ->  max_attempts=3 backoff_base_ms=150
t=1790108432  order_service->payment_service   rho=0.01  ->  max_attempts=3 backoff_base_ms=100
t=1790108434  order_service->payment_service   rho=0.03  ->  max_attempts=3 backoff_base_ms=100
t=1790108436  order_service->payment_service   rho=0.10  ->  max_attempts=3 backoff_base_ms=100
```

Retry budget cut as rho rises, restored as it falls

python3 experiment_three_arms.py

```
============================================================
RESULTS
============================================================

NORMAL TRAFFIC (no fault)
  policy                  success    avg latency      calls
  ------------------------------------------------------------
  no retries                88.5%         160ms        462
  always retry             100.0%         190ms        483
  adaptive controller       99.4%         185ms        492

DURING THE FAULT (payment_service slow)
  policy                  success    avg latency      calls
  ------------------------------------------------------------
  no retries                 0.7%        1337ms        475
  always retry               1.4%        3473ms       1036
  adaptive controller        1.7%        2353ms        747

AFTER THE FAULT IS REMOVED
  policy                  success    avg latency      calls
  ------------------------------------------------------------
  no retries                87.2%         152ms        460
  always retry             100.0%         186ms        493
  adaptive controller      100.0%         175ms        486

what to look at: during the fault, compare 'calls' (how hard the
system worked) against 'success' (what users actually got).
```

Output: Prediction Results

```
python3 prediction_model.py

built 645 training examples (34 positive / livelock-within-horizon)
temporal split: 451 train, 194 test (train is strictly earlier in time than test)

test set results:
  precision: 0.38
  recall:    1.00
  f1:        0.55
  confusion matrix [[TN, FP], [FN, TP]]:
  [[146, 30], [0, 18]]

feature weights (higher = pushes prediction toward 'livelock coming'):
  lambda_base                    -0.381
  lambda_retry                   -0.288
  rho                            0.504
  s_useful                       0.608
  concurrency                    0.405
  gateway->order_service         2.215
  order_service->payment_service -1.235
  order_service->inventory_service -0.971

control test -- same model, training labels shuffled:
  precision: 0.00  (chance level is about 0.09)
  recall:    0.00
  f1:        0.00
  if these are close to chance, the real model above was
  learning from the features, not leaking the label.
```

Recall 1.00 means no coming livelock was missed. The shuffled-label control scoring 0.00 is the evidence that the real result is not leakage.

Future Work

- Implement the TGN-Edge model now that the prediction task is shown to be learnable.
- Calibrate thresholds using a public production trace such as the Alibaba microservice dataset.
- Add the remaining two controller stability mechanisms.
- Extend to a deeper call chain, since retry amplification compounds with every additional layer.

Conclusion

- A working microservice testbed was built in which a retry storm can be triggered and measured.
- Livelock detection was implemented directly from the four formal conditions, and produced no false alarms on healthy traffic.
- Livelock can be predicted a few seconds in advance, and this was verified against a control test rather than assumed.
- An adaptive retry controller reduced wasted work by about 28 percent without losing the benefit of retries in normal conditions.
- All seven identified modules are implemented and can be demonstrated live.

Main finding: retries are helpful for short-lived faults and harmful during saturation. A fixed retry policy cannot be correct in both situations, which is why the control has to adapt at runtime.

Thank You